



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 7
«Реализация методов Штраусса и Винограда и их сравнение»
по курсу «Численные методы линейной алгебры»

Студент группы ИУ9-71Б Лысенко В. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Цель работы

Реализовать методы Штраусса и Винограда и сравнить их точность со стандартным алгоритмом вычисления произведения матриц. Сравнить алгоритмы по времени работы.

2 Задачи

1. Реализовать алгоритмы Штраусса и Винограда.
2. Сравнить точность методов со стандартным.
3. Изменить алгоритм Штраусса таким образом, чтобы для матриц с начальным размером более 64×64 вычисления проводились классическим образом, а в ином случае - неоптимальным.
4. Сравнить алгоритмы по времени работы и построить соответствующий график.

3 Практическая реализация

Код программы:

```
using LinearAlgebra
using Plots

function standard(A, B)
    n = size(A, 1)
    C = zeros(Float64, n, n)
    for i in 1:n
        for j in 1:n
            for k in 1:n
                C[i, j] += A[i, k] * B[k, j]
            end
        end
    end
    return C
end

function winograd(A, B)
    n = size(A, 1)
    C = zeros(Float64, n, n)
    m = n ÷ 2

    for i in 1:n
        for j in 1:n
            for k in 1:m
                C[i, j] += (A[i, 2*k-1] + B[2*k, j]) * (B[2*k-1, j] + A[i, 2*k])
                C[i, j] -= A[i, 2*k-1] * A[i, 2*k]
                C[i, j] -= B[2*k, j] * B[2*k-1, j]
            end
        end
    end
    return C
end

function strassen(A, B)
    n = size(A, 1)
    if n <= 64
        return strassen_false(A, B)
    end
    if n > 64
        return strassen_true(A, B)
    end
end
```

```

end

function strassen_false(A, B)
    n = size(A, 1)
    if n <= 2
        return standard(A, B)
    end
    if n % 2 != 0
        new_size = n + 1
        A_new = zeros(Float64, new_size, new_size)
        B_new = zeros(Float64, new_size, new_size)
        A_new[1:n, 1:n] = A
        B_new[1:n, 1:n] = B
        C_new = strassen_false(A_new, B_new)
        return C_new[1:n, 1:n]
    end

    k = n ÷ 2

    A11 = A[1:k, 1:k]
    A12 = A[1:k, k+1:n]
    A21 = A[k+1:n, 1:k]
    A22 = A[k+1:n, k+1:n]

    B11 = B[1:k, 1:k]
    B12 = B[1:k, k+1:n]
    B21 = B[k+1:n, 1:k]
    B22 = B[k+1:n, k+1:n]

    P1 = strassen_false(A11 + A22, B11 + B22)
    P2 = strassen_false(A21 + A22, B11)
    P3 = strassen_false(A11, B12 - B22)
    P4 = strassen_false(A22, B21 - B11)
    P5 = strassen_false(A11 + A12, B22)
    P6 = strassen_false(A21 - A11, B11 + B12)
    P7 = strassen_false(A12 - A22, B21 + B22)

    C11 = P1 + P4 - P5 + P7
    C12 = P3 + P5
    C21 = P2 + P4
    C22 = P1 + P3 - P2 + P6

    C = zeros(Float64, n, n)
    C[1:k, 1:k] = C11
    C[1:k, k+1:n] = C12
    C[k+1:n, 1:k] = C21

```

```

    C[k+1:n, k+1:n] = C22

    return C
end

function strassen_true(A, B)
    n = size(A, 1)
    if n <= 64
        return standard(A, B)
    end
    if n % 2 != 0
        new_size = n + 1
        A_new = zeros(Float64, new_size, new_size)
        B_new = zeros(Float64, new_size, new_size)
        A_new[1:n, 1:n] = A
        B_new[1:n, 1:n] = B
        C_new = strassen_true(A_new, B_new)
        return C_new[1:n, 1:n]
    end

    k = n ÷ 2

    A11 = A[1:k, 1:k]
    A12 = A[1:k, k+1:n]
    A21 = A[k+1:n, 1:k]
    A22 = A[k+1:n, k+1:n]

    B11 = B[1:k, 1:k]
    B12 = B[1:k, k+1:n]
    B21 = B[k+1:n, 1:k]
    B22 = B[k+1:n, k+1:n]

    P1 = strassen_true(A11 + A22, B11 + B22)
    P2 = strassen_true(A21 + A22, B11)
    P3 = strassen_true(A11, B12 - B22)
    P4 = strassen_true(A22, B21 - B11)
    P5 = strassen_true(A11 + A12, B22)
    P6 = strassen_true(A21 - A11, B11 + B12)
    P7 = strassen_true(A12 - A22, B21 + B22)

    C11 = P1 + P4 - P5 + P7
    C12 = P3 + P5
    C21 = P2 + P4
    C22 = P1 + P3 - P2 + P6

    C = zeros(Float64, n, n)

```

```

        C[1:k, 1:k] = C11
        C[1:k, k+1:n] = C12
        C[k+1:n, 1:k] = C21
        C[k+1:n, k+1:n] = C22

        return C
end

n = 100
A = randn(n, n)
B = randn(n, n)

standard_arr = standard(A, B)
winograd_arr = winograd(A, B)
strassen_arr = strassen(A, B)

error_winograd = maximum(abs.(standard_arr .- winograd_arr))
error_strassen = maximum(abs.(standard_arr .- strassen_arr))

println("Погрешность алгоритмаВинограда ")
println(error_winograd)
println("Погрешность алгоритмаШтрасса ")
println(error_strassen)

N = 2:4:400
standard_times = Float64[]
winograd_times = Float64[]
strassen_times = Float64[]

for n in N
    A = randn(n, n)
    B = randn(n, n)

    standard_t = @elapsed standard(A, B)
    push!(standard_times, standard_t)

    winograd_t = @elapsed winograd(A, B)
    push!(winograd_times, winograd_t)

    strassen_t = @elapsed strassen(A, B)
    push!(strassen_times, strassen_t)
end

plot(N, standard_times, label="Стандартный алгоритм", xlabel="Размер матрицы N",
    , ylabel="Время c()", legend=:topleft)

```

```

|| plot!(N, winograd_times, label="Алгоритм Винограда")
|| plot!(N, strassen_times, label="Метод Штрассена")

```

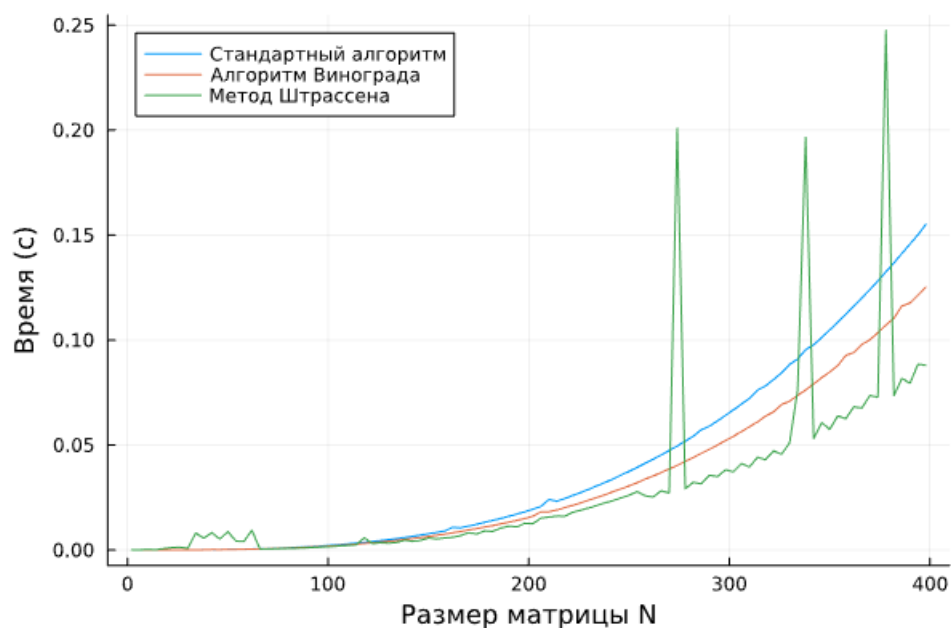
4 Результаты

Результат выполнения программы:

```

Погрешность алгоритма Винограда
4.263256414560601e-14
Погрешность алгоритма Штрасса
3.4638958368304884e-14

```



5 Выводы

В ходе выполнения лабораторной работы были реализованы и сравнены по точности и времени выполнения алгоритмы Штраусса и Винограда по вычислению произведения матриц размера $n \times n$. В результате алгоритм Штраусса показал себя наиболее эффективным и точным. На графике также наглядно видно, почему на моменте разбиения матриц размерами меньше 64×64 метод Штраусса начинает проводить вычисления по стандартному алгоритму.